glineRestApi v2 — User Guide VERSION 2

Module: glineRestApi 2.4 (Niagara 4.14) · **Default port:** 8082 · **Vendor:** Shanghai Gline Net Co.

■ **Documented version: v2.4.0** — module `glineRestApi-rt`, `vendorVersion` **2.4.0**. This document describes exactly one build; the build is pinned by hash so it cannot be confused with any other release.

Artifact	Size	SHA-256
`glineRestApi-rt.jar` (signed)	38390 B	`efafe1e60096118ce1d768352c1088a4e3a5ff8c3c34d48f4b8aff6f3744f833`

MD5 `d97c7c604eb8b19152276d3ce723939d` · SHA-1 `837d4b81c22ae000a6d9f797f750c9bb51029062`
Version and hashes are the authoritative record for this document. Any build with a different `vendorVersion` or hash is a different release — do not assume it behaves as described here.

■ **This document covers version 2 only.** v2 is a separate module, distributed on request. For the free unauthenticated module, see the separate **v1 guide**.

Table of Contents

1. Overview & Architecture
2. Authentication (Basic Auth)
   2.3 Service properties
3. REST API Endpoints
   3.1 GET /api/status
   3.2 GET /api/modules *(stub)*
   3.3 GET /api/types *(stub)*
   3.4 GET /api/config
   3.5 POST /api/batch (12 Operations + BQL)
   3.6 Program endpoints
4. Facet editing (v2.4)
5. Alarm & history extensions via REST
6. Reading alarms & history — Web Box
7. Verified test samples
8. Deployment
9. Known limitations

1. Overview & Architecture

glineRestApi is the v2 module. It keeps the v1 component-mutation API and adds a complete **security, query, and editing layer**.

Architecture

```
RestApiServiceV2  (extends BComponent)
 ├─ enabled  (boolean)
 ├─ port     (int, default 8082)
 ├─ authUser / authPass          (Basic Auth credentials)
 ├─ allowCidr                     (comma-separated allow-list)
 ├─ compileHome / javacPath       (program compile toolchain)
 ├─ apiStatus                     (read-only; last server state)
 ├─ started() → Creates HttpServer
 └─ stopped() → Stops  HttpServer

Handlers:
  Status   → GET  /api/status          (auth-exempt)
  Modules  → GET  /api/modules         (informational stub)
  Types    → GET  /api/types           (informational stub)
  Config   → GET  /api/config?path=/xxx
  Batch    → POST /api/batch
  Program  → /api/program/{write,addslot,status,compile}
```

- JDK built-in HttpServer — no external container.
- JSON request/response.
- **Every request must be authenticated** (HTTP Basic Auth) — see Section 2.
- Batch mutation model: all writes go through `POST /api/batch`.

2. Authentication (Basic Auth)

The REST API is **authenticated by default**. Every request must carry HTTP Basic credentials; a missing or wrong credential is rejected with **HTTP 401**.

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[{"OP":"list","PATH":"/"}]'
```

The **only** endpoint exempt from authentication is `GET /api/status`, usable as a liveness probe for monitoring:

```
# Unauthenticated liveness probe — the sole auth-exempt endpoint
curl http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

Credentials are configurable at runtime on the service component and persist across station restarts. A **CIDR allow-list** can further restrict access to trusted network segments; a request from outside the list is rejected with **HTTP 403** (loopback is always allowed as a recovery path).

Request	Result	Note
No credentials	**401**	Rejected
Wrong credentials	**401**	Rejected
Valid credentials, outside allow-list	**403**	Rejected
Valid credentials, inside allow-list	200	Accepted
`GET /api/status` without credentials	200	Liveness probe — the only exemption

■ **Even though the API is authenticated, do not expose the port directly to the public internet — put it behind a firewall or reverse proxy.**

2.3 Service properties

The service component exposes these configurable properties (all persist across station restarts). Credentials and the allow-list are covered in Section 2.1.

Property	Type	Purpose
`enable`	boolean	Start / stop the embedded HTTP server
`port`	int	Listen port (default **8082**)
`authUser` / `authPass`	string	Basic Auth credentials — empty `authUser` disables the check
`allowCidr`	string	Comma-separated CIDR allow-list; empty means no restriction
`compileHome`	string	Niagara install root used by `/api/program/compile`
`javacPath`	string	Path to the Java compiler used by `/api/program/compile`
`apiStatus`	string	**Read-only.** Last reported server state (e.g. `OK listening 8082`)

3. REST API Endpoints Reference

3.1 GET /api/status

Service liveness. This is the only endpoint that does not require credentials.

```
curl http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

The port is configurable on the service component; `8082` is the default. Replace `<HOST>:8082` with your station's actual host and port.

3.2 GET /api/modules

Informational stub. This endpoint does **not** enumerate modules — it returns a fixed hint pointing at `/api/config` . (In v1 this endpoint returned the real module list.)

```
curl -u "<USER>:<PASS>" http://<HOST>:8082/api/modules
{"note":"use /api/config to browse station tree"}
```

3.3 GET /api/types

Informational stub. Returns a fixed hint listing common types; it does **not** enumerate the type dictionary. To enumerate types, browse the station tree via `/api/config` or run a BQL query (Section 3.5).

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/types?module=control"
{"note":"use /api/config to browse types; common types: baja:Folder, ..."}
```

3.4 GET /api/config

Browse the station configuration tree at any path.

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/config?path=/"
{"name":"station","type":"Station",
 "slots":[{"name":"stationName","type":"String"},
          {"name":"Services","type":"ServiceContainer"}, ...],
 "children":[{"name":"Services","type":"ServiceContainer"}, ...]}
```

In v2 the `slots` member is an **array** of `{name, type}` entries (v1 returned an object). `children` lists immediate child components.

3.5 POST /api/batch — Core Operation

All mutations go through this endpoint. Send a JSON array of operation objects executed sequentially in order.

Common Fields

Field	Type	Description	Applies To

OP	string	Operation type	ALL
PATH	string	Component path	ALL
T	string	Type module:Type	add
N	string	Name / slot	add, mod, link
SLOT	string	Read that slot's facets	read
NEW	string	New name	rename, cp
V	any	Value / facet spec	mod, facet
TARGET	string	Target path	link, unlink
TARGET_N	string	Target slot	link
VAL	any	Invoke value	invoke
VAL_TYPE	string	num / bool / str	invoke
TARGET_PATH	string	Copy destination	cp
Q	string	BQL query text	bql

12 Operations + BQL

Op	Action	Example
1. ADD	Create component	`{"OP":"add","PATH":"/","T":"baja:Folder","N":"myFolder"}`
2. LINK	Connect	`{"OP":"link","PATH":"/src","N":"out","TARGET":"/tgt","TARGET_N":"in10"}`
3. RENAME	Rename (preserves links)	`{"OP":"rename","PATH":"/folder/old","NEW":"new"}`
4. CP / DUPLICATE	Deep copy + rebuild links	`{"OP":"cp","PATH":"/folder","NEW":"copy","TARGET_PATH":"/"}`
5. MOD	Write property	`{"OP":"mod","PATH":"/device","N":"ipAddress","V":"<IP>"}`
6. UNLINK	Remove link	`{"OP":"unlink","TARGET":"/tgtPoint"}`
7. REMOVE	Delete component	`{"OP":"remove","PATH":"/parent/child"}`
8. READ	Read property	`{"OP":"read","PATH":"/device","N":"ipAddress"}`
9. LIST	List children	`{"OP":"list","PATH":"/myFolder"}`
10. INVOKE	Call action (alias `action`)	`{"OP":"invoke","PATH":"/point/set","VAL":"42","VAL_TYPE":"num"}`
11. FACET	Write facets (3.5.1)	`{"OP":"facet","PATH":"/pt","N":"facets","V":"min=0;max=100"}`
12. SETFACET	Synonym for FACET (same handler)	`{"OP":"setfacet",...}`
+ BQL	Query → JSON table (3.5.2)	`{"OP":"bql","Q":"select name from control:NumericWritable"}`

3.5.1 BQL Query → JSON

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" \
  -d '[{"OP":"bql","Q":"select name,slotPath from control:NumericWritable"}]'

[{"OP":"bql","ord":"station:|slot:/|bql:select name,slotPath from control:NumericWritable",
  "columns":["name","slotPath"],
  "rows":[{"name":"twoOnOffsmartSwitch$2eswitchCtrl","slotPath":"..."}, ...]}]
```

The query text may be plain BQL (`Q`) or a full ord. Optional parameters:

Field	Type	Purpose
`Q`	string	The BQL query text (or a full `station:|slot:…|bql:…` ord)
`SCOPE`	string	Slot path the query runs against — default `/`
`LIMIT`	int	Maximum rows returned — default **200**

Returns a structured table (`columns` + `rows`) ready for reports, dashboards, or AI pipelines — no XML parsing.

3.6 Program Endpoints

Endpoint	Method	Purpose
`/api/program/write`	POST	Write program source
`/api/program/addslot`	POST	Add a program slot
`/api/program/status`	GET	Read class, status, checksum, source length

`/api/program/compile`	POST	Compile the program

On v1 these endpoints do not exist (HTTP **404**).

4. Facet Editing (v2.4)

Write **facets** on any property or point through `MOD` or `FACET`, and read a specific slot's facets back.

```
# Numeric facets — min / max / units / precision
{"OP":"facet","PATH":"/Folder/Point","N":"facets","V":"min=0;max=100;units=degC;precision=2"}

# Enum facets — plain or indexed form
{"OP":"facet","PATH":"/Folder/Point","N":"facets","V":"range=Low,Mid,High"}
{"OP":"facet","PATH":"/Folder/Point","N":"facets","V":"range=0:Low,1:Mid,2:High"}

# Read a specific slot's facets
{"OP":"read","PATH":"/Folder/Point","SLOT":"out"}
```

Supported: numeric facets (`min` / `max` / `precision` / `fieldWidth`), `units` , `trueText` / `falseText` , and enum `range=` in both plain and indexed forms. The facet spec is split on `;` first, so `range=` values may contain commas.

5. Alarm & History Extensions via REST

Alarm and history capture is configured by attaching **extensions** to a point. These are ordinary components, so v2 can create, configure, and **activate** them over HTTP — no Workbench required.

Extension type	Purpose
`alarm:AlarmSourceExt`	Alarm source extension
`history:NumericCovHistoryExt`	Numeric history, change-of-value
`history:BooleanCovHistoryExt`	Boolean history, change-of-value
`history:EnumCovHistoryExt`	Enum history, change-of-value
`history:StringCovHistoryExt`	String history, change-of-value
`history:NumericIntervalHistoryExt`	Numeric history, fixed interval

Attach a history extension and activate it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"history:NumericCovHistoryExt","N":"he"},
{"OP":"mod","PATH":"/myFolder/pt/he","N":"enabled","V":true}
]'
# → enabled=true, active=true — history collection is live
```

Attach an alarm extension and configure it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"alarm:AlarmSourceExt","N":"ae"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"alarmClass","V":"/AlarmClass"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"timeDelay","V":5000}
]'
```

Known MOD limits for extensions

Slot	Behaviour
`enabled`	Boolean — writable; activates a history extension
`alarmClass`	String — writable
`timeDelay`	`RelTime` — write a **raw millisecond number** (`5000`), not `"5s"`
`historyName` / `toOffnormalText`	`BFormat` — **cannot** be written via MOD
`alarmEnable`	`AlarmTransitionBits` — **cannot** be written via MOD

Extensions created via REST remain fully visible and editable in Workbench. Reading the collected data is covered in Section 6.

6. Reading Alarms & History — Web Box

The REST API has no alarm or history **query** endpoints by design. That data is read through the station's **Web Box** feature, which is automatable.

Endpoint	Method	Purpose
`/wsbox/query/data/<ord>`	GET	Read history records for a point (paged, sortable)

`/wsbox/query/source/<ord>`	GET	History point source / metadata
`/wsbox/query/lastTime/<ord>`	GET	Timestamp of the most recent record

Query parameters

- `offset` / `limit` — pagination window.
- `sortColumn=timestamp` and `descending=false` — chronological order.
- `timeRange=startTime=…;endTime=…` — bounded time window.

Slot path encoding

History points are addressed by an **ord** of the form `history:/<station>/<historyName>`. In a URL it must be escaped — the mapping includes `:` → `$3a`, `/` → `$2f`, `?` → `$3f`, `=` → `$3d`, `&` → `$26`, `$` → `$24`.

> ■ **Escaping `&` is mandatory.** A raw `&` is parsed as an HTTP parameter separator and the query silently returns an empty result set.

Automation notes

- Requests must be sent as **GET**.
- The response is a chunked **NDJSON** stream — read it with a streaming client (e.g. browser `XMLHttpRequest` with `onprogress`), not a single blocking read.
- The CSRF token must be obtained inside the station page context (`csrfUtil.getCsrfToken()`) and sent in the `x-niagara-csrfToken` header; the station session must already be authenticated.

With these pieces in place, alarm and history extraction can be fully automated.

7. Verified Test Samples

All operations were tested on a Niagara 4.14 station with default admin credentials.

Modbus TCP Device

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork","T":"modbusTcp:ModbusTcpDevice","N":"T8600_new"},
{"OP":"mod","PATH":"/.../T8600_new","N":"deviceAddress","V":2},
{"OP":"mod","PATH":"/.../T8600_new","N":"ipAddress","V":"<IP>"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"OnOff"},
{"OP":"add","PATH":"/.../OnOff","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../OnOff/proxyExt","N":"dataAddress","V":"2|decimal"},
{"OP":"invoke","PATH":"/Drivers/ModbusTcpNetwork/T8600_new/ping"}]'
```

Add + Link + Rename

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"testFolder"},
 {"OP":"add","PATH":"/testFolder","T":"control:NumericWritable","N":"src"}, ...]
```

Result: the link survives the rename.

Deep Copy (CP)

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"original"}, ...]
```

Result: internal links are rebuilt in the copy.

Full Lifecycle

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"lifecycle"}, ...]
```

Result: unlink removes the Link child; remove deletes the source.

Verification: 40 / 40 test cases passed (29 functional + 11 access-control).

8. Deployment

1. Copy the v2 module JAR into the station host's `modules/` directory.
2. Restart the station so the module loads.
3. Add the **RestApiServiceV2** service, set the port (default **8082**), username, and password.
4. Optionally set the CIDR allow-list.
5. The API is then live on the configured port; every request needs Basic Auth.

Dependencies

baja 4.14 · nre 4.14 · control 4.14 · modbusCore 4.14 · modbusTcp 4.14

9. Known Limitations

Limitation	Detail

Authentication required	Every request needs valid Basic Auth credentials; only `GET /api/status` is exempt. Do not expose the port directly to the public internet.
Enum Properties	regType, dataType: cannot write via MOD.
BFacets Properties	deviceFacets: cannot write via MOD.
Non-scalar slots	`BFormat` and `AlarmTransitionBits` slots cannot be written via MOD (property-type limit, not an API limit).
Modules / Types are stubs	`/api/modules` and `/api/types` return fixed hints, not real listings. Use `/api/config` or BQL.
No alarm/history query	No alarm or history query endpoint by design. Configure extensions via REST (Section 5) and read data via Web Box (Section 6).
Disable needs restart	Disabling the service requires a station restart for the module to unload.

For commercial customization, integration, or deployment support, please contact:

Shanghai Gline Net Co.
Email: jason.zhang@gline-net.com
Tel / WhatsApp: +86 138 0199 0968

© 2026 Shanghai Gline Net Co. — glineRestApi v2 — End of Document